ResQAI: AI-Based Accident Detection, Severity Assessment and Intelligent Emergency Response System

Adepu Meenakshi
Roll No. 23K91A0503
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Ayila Shanmukha Sai
Roll No. 23K91A0521
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Chekka Chandrashekhar
Roll No. 23K91A0552
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Avula Sreeya
Roll No. 23K91A0519
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Shirisha Reddy
[DESIGNATION], project guide
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**A crash on a quiet road can go unreported for a long time, and today's crash alerts usually say only that an impact occurred, not how serious it was. This paper presents ResQAI, a web application that turns motion-sensor readings (accelerometer, gyroscope, speed and jerk) into a graded, located emergency alert. Two random forests detect a crash and grade it as none, minor, moderate or severe; only a moderate or severe result opens a 15-second confirmation in which the occupant can cancel a false alarm; otherwise the browser location is read, the three nearest hospitals are ranked by great-circle distance and an alert message is composed. No real recordings were available, so the models were trained and tested on 12,000 synthetic events. On the 2,400 held-out events the detector scored 1.000 on every metric, which a one-line threshold also achieves, and severity accuracy was 0.9996 (five-seed mean 0.9988). Events the generator never produces, such as potholes, were flagged as accidents about half the time. The alert path took 56 ms on loopback, excluding the confirmation window; nothing is transmitted. The pipeline works end to end; it is not shown to perform this well on real roads.

***Index Terms*---**accident detection, emergency response, hospital recommendation, random forest, severity prediction, smartphone sensors.

I. INTRODUCTION

Road traffic injury is a leading cause of death worldwide [2], and the outcome of a serious crash depends heavily on how soon help arrives. Victims are often unable to call for help, and on a remote stretch an accident may go unreported until a passer-by stops. Modern smartphones carry accelerometers, gyroscopes and GPS, so they are a low-cost crash sensor, but the alert built on that sensor is usually binary: an impact happened. Without a severity grade, a responder cannot tell a minor scrape from a critical collision, and without a confirmation step, a dropped phone or a hard stop can dispatch a false alarm.

Research on severity has mostly worked after the event, on recorded crash reports. The base paper [1] predicts injury, fatal and property-loss severity from 30,442 police-reported crashes with a multi-task FT-Transformer [3], [4]; it never detects a crash, never sees a sensor and never contacts anyone. Prototypes that do detect crashes, from Arduino boards with a GSM modem [5], [6] to camera-based systems [7], rarely grade severity or report measured error rates. The loop from live sensing through severity grading and confirmation to a located, prioritised alert is therefore not closed in the work surveyed.

ResQAI closes that loop at demonstration scale. Its contributions are: (1) a detector and a four-level severity model (random forests) over an eleven-feature sensor reading, gated so that a non-crash is never graded; (2) a confirmation rule that only a moderate or severe result starts a 15-second window for the occupant to cancel, followed by GPS location and nearest-hospital ranking; (3) a measured evaluation that goes beyond one accuracy figure, with a five-seed repeat, baselines, a feature ablation, a noise sweep, out-of-distribution probes, latency and a 55-check test run; and (4) a plain record of what the evaluation could not show. Section II reviews related work, Section III describes the method, Section IV gives the results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop PC: Windows 11 Pro, an Intel Core i5-12500 processor, 15.7 GB of RAM and no GPU. During the runs other programs held most of the memory, so timings are conservative.

B. Software Requirements

Python 3.11.9, scikit-learn 1.5.0 [8] with NumPy 1.26.4 and pandas 2.2.1; Flask 3.1 for the API with scrypt password hashing and signed tokens; React 19 and Vite 8 with Leaflet and Recharts for the console, and Node.js 18 or later. The shipped model files were written by scikit-learn 1.9.0 and loaded under 1.5.0; a retrain with the same seed agrees with them on every test event.

II. LITERATURE SURVEY

Base paper. Wang, Liang and He [1] predict three outcomes of a traffic accident (injury severity in 5 classes, fatal-accident severity in 5 classes and property loss in 6 classes) from police-reported records of Dongguan, China,

2019-2023: 30,442 records split into 19,483 for training, 4,871 for validation and 6,088 for testing, with driver, vehicle, road and environment attributes. A multi-task FT-Transformer [3] embeds each tabular feature as a token and shares them across the three tasks, with a loss that weights each task by its uncertainty [4]; SMOTE balances the training set and SHAP explains the predictions. It reaches accuracies of 70.31%, 90.38% and 80.79% (F1 64.48%, 86.71% and 79.18%) and beats eight baselines including XGBoost, LightGBM, CatBoost and a random forest. The authors note a single city, proprietary data, injury proxied by the number of injured and no ordinal modelling. The model describes a crash that has already been reported; it does not detect one.

Sensor and IoT detection. GreenBox [5] uses an Arduino Uno and a SIM7600G-H GSM and GPS shield with four push-button impact sensors, an MPU-6050 for rollover beyond 70 degrees, water sensors and an engine-cut relay, and sends an SMS with location and speed to preset contacts; the owner can cancel a false detection by SMS. It is evaluated qualitatively (alerts "in a matter of seconds") with no accuracy or latency figure, and it applies no learning. GreenBox itself surveys earlier IoT designs [6], [9], [10], [11], [12]: accelerometer, vibration or heart-rate sensors with GPS and GSM alerts and, in one case, an alcohol sensor and a black-box log; it points out that accelerometer-only detection risks false positives on sudden braking. None of them grades severity.

Vision-based detection. CIRS [7] is a multi-agent framework in which YOLOv11 detects accidents in camera frames (5,200 accident and 4,800 non-accident frames), a vision-language model describes the scene, and further agents locate and dispatch. YOLOv11 reaches top-1 accuracy 86.5%, precision 0.866, recall 0.856 and F1 0.861, and 128 of 1,044 non-accident frames were falsely classified as accidents. It needs fixed cameras, which a vehicle on a rural road does not have.

Severity prediction. Random forests and boosting on crash records have been applied to teen-driver crashes [13] and to logistic regression, decision-tree and forest comparisons on 9,472 Taiwanese crashes [14]; recurrent [15] and convolutional [16] networks were tried on larger records (the base paper cites 73.76% accuracy for the recurrent model); explainable deep learning has been used for risk prediction [17]; and a recent review compares such methods [18]. All of them learn from recorded attributes after the event.

Positioning. The surveyed systems sit at two ends. IoT prototypes sense the crash but treat every impact alike and measure little; record-based severity models grade well-defined outcomes but cannot act, because their input is the report of an accident that has already been handled. The base paper's own limits make the distance explicit: its labels come from police files, its injury severity is a count of injured persons, and its inference cost is not studied. A phone-based pipeline must work from a few seconds of motion data, tolerate noise and orientation, and keep false alarms rare, because each costs the occupant a countdown and, if sent, a responder's time. GreenBox's SMS cancellation [5] and the 128 false positives among 1,044 non-accident frames in CIRS [7] both show that the confirmation step and the false-alarm rate are central, which is why ResQAI measures them.

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	Multi-task FT-Transformer, SHAP	30,442 Dongguan crash records	Accuracy 70.31 / 90.38 / 80.79 %	Post-event; no sensing or alert
[5]	Arduino + GSM/GPS, SMS alert	Prototype tests	Qualitative, "seconds"	No metrics, no learning, no severity
[6], [10]	Accelerometer / vibration + GSM	Prototypes	Not reported here	No severity grade
[7]	YOLOv11 + vision-language agents	5,200 + 4,800 frames	Top-1 86.5 %, F1 0.861	Fixed cameras; no severity
[13], [14]	RF, XGBoost, logistic, tree	8,859 and 9,472 crashes	Forest or XGBoost best	Post-event records
[15], [16]	RNN, CNN on crash records	Crash records	RNN 73.76 % (as cited in base)	Post-event; opaque

Gap. Severity work learns from records written after the crash, and detection work seldom measures its error rates or grades severity. No surveyed system detects a crash from motion signals, grades it, lets the occupant cancel, and then locates the crash and ranks hospitals with reported measurements. ResQAI targets that gap at demonstration scale.

III. METHODOLOGY

Architecture. Fig. 1 shows the tiers. A React console calls a single Flask process that serves five scenario generators, the prediction route, the hospital ranking and the dispatch composition, with accounts handled by a separate blueprint. There is no database: users are stored as scrypt hashes in a JSON file, the ten-hospital directory is a second JSON file, and the scaler and the two forests are loaded once from four pickles. Readings, predictions and alerts exist only in requests and in the browser's state.

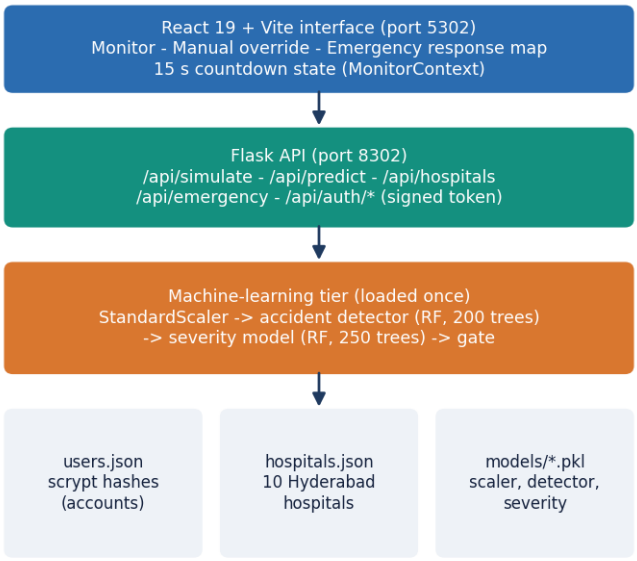


Fig. 1. ResQAI architecture: React console, Flask API, machine-learning tier and the file-based data tier.

Data. No real smartphone recordings were available, so the project generates 12,000 events from a seeded generator (seed 42): 7,289 normal-driving, 2,324 harsh-braking, 1,215 minor, 725 moderate and 447 severe crashes (19.9% accidents). A crash draws a random-direction acceleration spike of 15-28, 28-45 or 45-80 m/s2 with rising gyroscope noise, deceleration and jerk; braking draws a forward deceleration of about 4.5 m/s2. Each event has eleven features: three acceleration axes, three gyroscope axes, their two magnitudes, speed before the event, speed change and jerk. The split is 80/20 stratified on the accident label (9,600 training, 2,400 test events; 477 test accidents).

Models. Features are standardised, z = (x - mu) / sigma, with the training mean and deviation. The accident detector is a random forest [19] of 200 trees, depth 10, with balanced class weights, whose accident probability is the mean of the trees' class probabilities

$$P(\text{accident} \mid x) = (1/T)\,\Sigma_t\, p_t(\text{accident} \mid z), \quad T = 200, \qquad (1)$$

and the severity model is a second forest (250 trees, depth 12) over four classes. A forest was chosen because the inputs are eleven tabular numbers, training is seconds on a CPU, and the same family is a strong baseline in the base paper. The application then gates the grade so that a non-crash is never graded,

$$S(x) = 0 \text{ if the detector says no accident, otherwise the severity forest's class.} \qquad (2)$$

Training. Both forests use scikit-learn settings other than the tree count, depth and balanced class weights (random state 42). The severity forest is trained on all events with None as class 0, so it also learns what a non-crash looks like; the gate then makes the detector authoritative for the crash decision. The training script holds out 20% of the events and prints the accuracy and a classification report; it uses no validation split and no cross-validation, so the hyper-parameters were fixed in advance and not tuned. The evaluation below adds the repeats, baselines and stress tests that the script lacks.

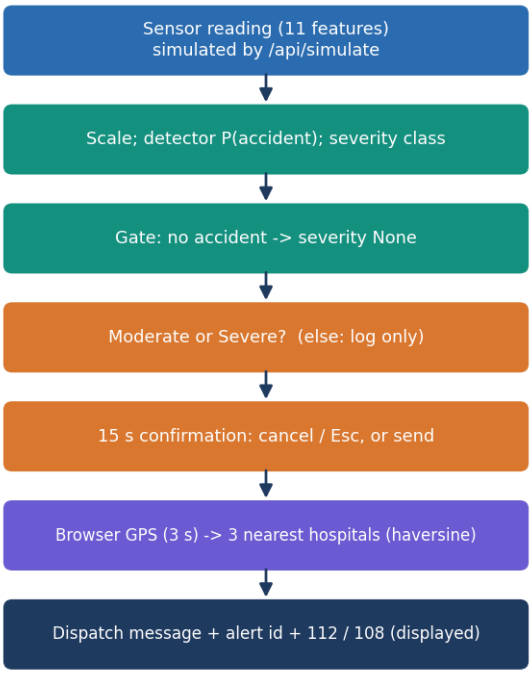


Fig. 2. Main flow of one event: detection, grading, gate, confirmation, location, hospital ranking and the displayed dispatch.

Confirmation and dispatch. The React console runs a state machine. A Moderate or Severe result pauses live monitoring (one reading every 1.5 s) and opens a 15-second countdown; the occupant can cancel with a button or Escape, which is logged as a false alarm, and at zero the alert is sent. The browser location is read with a 3-second timeout (otherwise the server uses a point near the default area), and the three nearest of ten hospitals are chosen by the haversine distance [20]

$$d = 2R\,\text{asin}(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2)}), \quad R = 6371\ \text{km}. \qquad (3)$$

The response carries an alert id, the coordinates, the nearest hospitals with phone numbers, a message with a map link, and the national (112) and ambulance (108) numbers. No SMS, call or e-mail is actually sent; the message is displayed. If the model files are missing, three thresholds on acceleration magnitude (15, 28 and 45 m/s2) give a rule-based grade instead.

Implementation. The API validates registration (a name of 2 to 60 characters, an e-mail address, a password of 8 to 128 characters) and stores scrypt password hashes. Tokens are signed with a 12-hour expiry and sent as a Bearer header; five failed logins lock an account for five minutes, and setting REQUIRE_AUTH=1 makes every route except health and authentication demand a token (it is off by default so the original single-page dashboard keeps working). The simulation route draws each scenario from the training generator's parameters. The prediction route fills missing fields with defaults (1 g on the vertical axis, zeros elsewhere), derives the magnitudes, takes the absolute jerk,

and returns the accident flag, the class, its label, the probability, the features used and the model source. The console keeps the last 40 sensor points and 25 events, offers a light and a dark theme, and shows the numbers 108 and 112 as tap-to-call links.

Reproducibility. Every figure and table in Section IV is produced by three scripts in the repository, one for the models (data regeneration with the training seed, baselines, ablations, noise, probes, thresholds), one for API latency and one for the unit, edge and security checks, and each writes a JSON file that this paper reads when it is built, so the text cannot drift from the measurements. The generator reproduces the training script's dataset exactly, so the held-out events are the ones the shipped models never saw.

Evaluation protocol. The shipped models were scored on the 2,400 held-out events (accuracy, precision, recall, F1, ROC-AUC, confusion matrices); the same procedure was repeated for five seeds with a new dataset, split and forests each time; logistic regression, a depth-10 tree, histogram gradient boosting and the three-threshold rule were trained on the same split; feature groups were removed and the models retrained; zero-mean Gaussian noise with a standard deviation of 0.05 to 1.0 times each feature's own was added to the test inputs; 1,000 events each of two constructed road-surface probes (pothole-like and speed-breaker-like, parameters in the repository, not real recordings) were scored; and latency was timed on 127.0.0.1 with 300 calls per route. The test run has 33 shipped smoke checks and 55 further unit, edge and security checks.

IV. RESULTS

Test conditions. Everything ran on the machine of Section I-A. The data are synthetic and drawn from the same generator for training and testing, so every accuracy below is an upper bound that says how well the classes are separated by construction. Table II lists the main figures.

TABLE II
MEASURED PERFORMANCE (SHIPPED MODELS, 2,400 HELD-OUT EVENTS)

Measure	Value
Detector accuracy / precision / recall / F1 / AUC	1.000 / 1.000 / 1.000 / 1.000 / 1.000
Detector confusion [[TN, FP], [FN, TP]]	[[1923, 0], [0, 477]]
Severity accuracy / macro-F1	0.99958 / 0.99866
Severity per-class F1 (None, Minor, Moderate, Severe)	1.000, 0.998, 0.997, 1.000
Alert trigger (Moderate or Severe): precision / recall	0.996 / 1.000; 0.046% false alarms
Five seeds: severity accuracy, mean ± SD	0.99875 ± 0.00066
Five seeds: severity macro-F1, mean ± SD	0.99501 ± 0.00293
Hard braking flagged as accident	0 of 460
Pothole-like / speed-breaker-like flagged (probes)	48.9% / 50.5%
Alert path (simulate + predict + emergency), mean (p95)	56.4 ms (82.7 ms)
POST /api/predict, mean (p95); throughput	26.2 ms (37.8 ms); 34.1 req/s

Detection. The detector made no error on 2,400 events and the lowest accident probability of a true crash (0.90) is well above the highest of a non-crash (0.17), so thresholds from 0.2 to 0.9 give identical, perfect results. This is not evidence of difficulty solved: braking and normal driving never exceed 13.2 m/s2 of acceleration magnitude, crashes never fall below 15.3, and the three-threshold rule detects as well as the forest (Table III).

At a threshold of 0.05 the detector raises 5 false alarms (precision 0.990) and at 0.95 it misses 2 crashes (recall 0.996); the plateau between is wide, so the operating point is not sensitive on this data. On the minor crashes, whose acceleration is the lowest of the crash classes, all 233 test events were flagged and 232 graded Minor.

TABLE III
SEVERITY GRADING AGAINST SIMPLER MODELS (SAME SPLIT)

Model	Severity accuracy	Macro-F1
Three-threshold rule on acc_magnitude	0.9750	0.9122
Logistic regression	0.9983	0.9946
Decision tree (depth 10)	0.9988	0.9960
Histogram gradient boosting	0.9996	0.9987
Random forest (ResQAI, retrained)	0.9996	0.9987

Severity. The only error in 2,400 events is a minor crash graded Moderate (acceleration 31.9 m/s2, in the overlap of the minor and moderate ranges); it is an adjacent-class error and turns a log-only event into an alert. Over five seeds the accuracy is 0.9988 ± 0.00066 and the worst seed is 0.9979. The forest beats the threshold rule (0.9996 against 0.9750) but is tied with gradient boosting, and logistic regression is already at 0.9983 (Fig. 3a). Acceleration magnitude carries most of the signal (permutation drop in macro-F1 0.46), followed by speed change and speed before; the gyroscope adds nothing measurable here, and acceleration magnitude alone gives severity accuracy 0.9858.

Alert-level view. What the occupant experiences is not the four-class grade but whether the countdown starts. On the test events 1 of 2156 events that should not start it did (a minor crash graded Moderate), and none of the 244 that should was missed. Across the five seeds, however, alert recall averages 0.9933 ± 0.0022 (worst 0.9914) and precision 0.9983, so about 0.7% of Moderate or Severe crashes are graded one step too low and would start no countdown, and the cost of that error is much larger than the cost of a false trigger. A deployment would therefore bias the alert rule toward recall, for example by also alerting on a Minor grade with a high probability and a large speed change.

TABLE IV
FEATURE-GROUP ABLATION (MODELS RETRAINED ON EACH SUBSET)

Features kept	Det. acc.	Sev. acc.	Sev. macro-F1
All 11	1.0000	0.9996	0.9987
Accelerometer only (4)	1.0000	0.9904	0.9622
acc_magnitude alone	1.0000	0.9858	0.9459
Without gyroscope	1.0000	0.9996	0.9987
Without speed features	1.0000	0.9983	0.9929
Without jerk	1.0000	0.9983	0.9938
Without acc_magnitude and jerk	0.9996	0.9917	0.9677

Removing any single group costs little because the features are correlated: magnitude, jerk and speed change all grow with the severity of the impact. Only dropping magnitude and jerk together lets one detector error appear. Gyroscope data would matter for rollover or spin, but the generator gives it only a noise level that rises with severity, so this experiment cannot show its value on real vehicles.

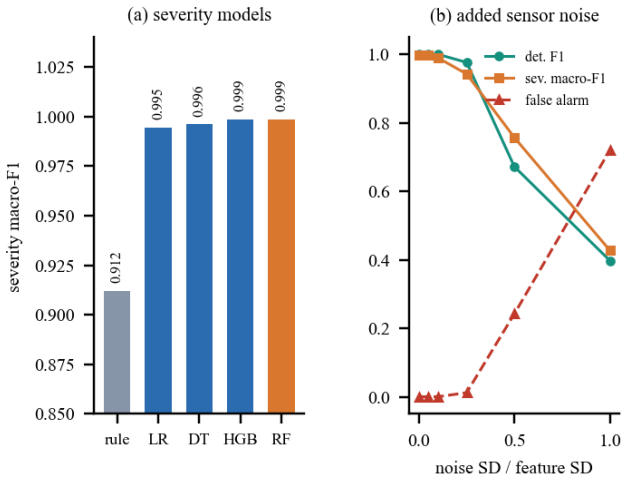


Fig. 3. (a) Severity macro-F1 of the rule, logistic regression (LR), decision tree (DT), gradient boosting (HGB) and random forest (RF). (b) Effect of sensor noise on the shipped models.

Robustness and failure cases. The models were never trained on noisy inputs. With noise at 0.1 of each feature's deviation they are unchanged (detector F1 1.000); at 0.25 the detector false-alarm rate is 1.2%; at 0.5 it is 24.2% and severity macro-F1 falls to 0.76 (Fig. 3b). The pothole-like and speed-breaker-like probes, which the generator never produces, were flagged as accidents 48.9% and 50.5% of the time, with mean probability about 0.5, so the detector has no concept of road-surface events; none reached Moderate or Severe, so none would start the countdown. The stated objective of separating potholes and speed breakers from crashes is therefore not demonstrated, only hard braking is. The shipped smoke test passes 33 of 33 checks; the 55 further checks pass 47, and the failures are input-validation gaps: non-numeric fields and a JSON array return HTTP 500, a negative hospital count returns nine of ten hospitals, out-of-range coordinates are accepted, and two alerts raised within the same second share one alert id.

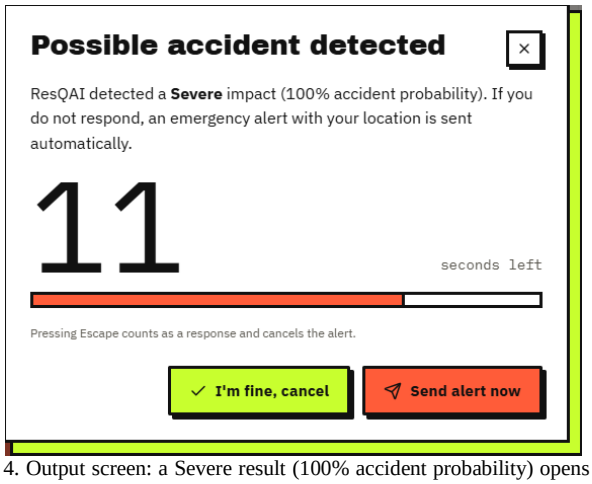


Fig. 4. Output screen: a Severe result (100% accident probability) opens the 15-second confirmation with Cancel and Send buttons.

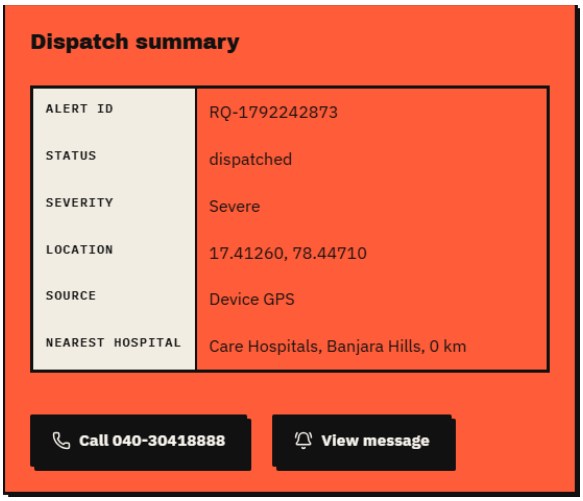


Fig. 5. Output screen: dispatch summary after the alert is sent: id, severity, device-GPS location and nearest hospital with its phone number.

Latency. On the loopback interface of the test machine the in-process model call takes 14.0 ms on average, /api/predict 26.2 ms and the three-call alert path 56.4 ms (p95 82.7 ms) on the Flask development server. This excludes the 15-second confirmation window by design and any real network delivery, which was not implemented and so not measured; no load test with several users was run.

Threats to validity. The generator, the probes and the hospital directory were written by the same team that built the models, which favours the system. The ten-hospital directory is a demonstration list for one city, and neither the GPS accuracy nor the map were scored. Alerts are not stored on the server and carry no user id, no concurrent-user test was run, the development server is single-process, and no occupant, clinician or responder was consulted. A real deployment would need labelled phone recordings with realistic noise, orientation and mounting, which the sensor streams in the console only imitate.

Confirmation window. The 15-second value is a design constant that was neither tuned nor tested with users: a shorter window risks dispatching for an occupant who is conscious but slow to respond, and a longer one delays help for one who is not. While the dialog is open live monitoring is paused, and a second event is logged but does not restart or replace the countdown. The window is also the dominant delay in the system: it is about 265 times the 56 ms of computation in the alert path.

Comparison with the base paper. The base paper [1] reports 70.31%, 90.38% and 80.79% accuracy for three severity tasks on 30,442 real crash records with up to six classes per task. ResQAI's figures are on synthetic sensor events, with a four-class severity and a binary detection. The datasets, the inputs (recorded attributes against sensor readings), the labels and the tasks all differ, so the numbers cannot be compared and no superiority is claimed; the base paper's results on real data are much harder-won than 0.9996 on generated events. What ResQAI adds is the part the base paper leaves out: live sensing, a confirmation step, location and a dispatch message, with measured latency.

V. CONCLUSION

ResQAI detects a crash from eleven sensor features, grades its severity, asks the occupant to confirm within 15 seconds, and composes a located alert with the three nearest hospitals. Of the six project objectives, severity grading (0.9996 accuracy on synthetic data), location with hospital ranking, and the evaluation of accuracy, false alarms and latency are achieved; collecting real smartphone data and separating potholes and speed breakers from crashes are not; and automated dispatch is only simulated, because no message is sent. The key finding is that the synthetic classes are separable enough that a one-line threshold detects as well as the forest, so the learned model matters only for severity, and that road-surface events outside the generator are flagged about half the time. The limits are the synthetic data, a single machine, no real GPS or messaging path and the validation gaps above. A pipeline of this shape is worth building on only once it has been trained and tested on real recordings; until then its numbers describe the software, not the roads.

Future work: record labelled data from real phones, including potholes and speed breakers; train and test on noisy, orientation-varied signals; add an SMS or call gateway and a live hospital directory; validate the inputs and issue unique alert ids; and tie each alert to an account so that contacts can be notified.

REFERENCES

[1] Y. Wang, Z. Liang and Y. He, "Multi-Task FT-Transformer: A high-performance and interpretable framework for traffic accident severity prediction," *IEEE Access*, vol. 14, pp. 21585-21606, 2026, doi: 10.1109/ACCESS.2026.3658687.

[2] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: WHO, Dec. 2023.

[3] Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko, "Revisiting deep learning models for tabular data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, Dec. 2021, pp. 18932-18943.

[4] A. Kendall, Y. Gal and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2018, pp. 7482-7491.

[5] J. Rocha, S. Soares, A. Valente and F. C. Pinto, "GreenBox: Prototyping of an automatic road accident detection system with real-time notification SMS," arXiv:2606.16468 [eess.SY], Jun. 2026.

[6] M. Karthik, L. Sreevidya, K. Vinodha, M. Thangaraj, G. Hemalatha and T. Viswak Sena, "Automatic messaging system by detecting the road accidents for vehicle applications," *Mater. Today: Proc.*, vol. 80, pp. 3124-3128, 2023.

[7] S. Ayesha, A. Aslam, M. H. Zaheer and M. B. Khan, "CIRS: A multi-agent machine learning framework for real-time accident detection and emergency response," *Sensors*, vol. 25, no. 18, Art. no. 5845, Sep. 2025, doi: 10.3390/s25185845.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[9] C. Dashora, P. E. Sudhagar and J. Marietta, "IoT based framework for the detection of vehicle accident," *Cluster Comput.*, vol. 23, no. 2, pp. 1235-1250, 2020.

[10] S. Sharma and S. Sebastian, "IoT based car accident detection and notification algorithm for general road accidents," *Int. J. Elect. Comput. Eng. (IJECE)*, vol. 9, no. 5, pp. 4020-4026, 2019.

[11] S. Uma and R. Eswari, "Accident prevention and safety assistance using IoT and machine learning," *J. Reliable Intell. Environ.*, vol. 8, no. 2, pp. 79-103, Jun. 2022.

[12] M. Anil Kumar, M. Venkata Suman, Y. Misra and M. Geetha Pratyusha, "Intelligent vehicle black box using IoT," *Int. J. Eng. Technol.*, vol. 7, no. 2.7, pp. 215-218, Mar. 2018.

[13] C. Lin, D. Wu, H. Liu, X. Xia and N. Bhattarai, "Factor identification and prediction for teen driver crash severity using machine learning: A case study," *Appl. Sci.*, vol. 10, no. 5, p. 1675, Mar. 2020.

[14] M.-M. Chen and M.-C. Chen, "Modeling road accident severity with comparisons of logistic regression, decision tree and random forest," *Information*, vol. 11, no. 5, p. 270, May 2020.

[15] M. Sameen and B. Pradhan, "Severity prediction of traffic accidents with recurrent neural networks," *Appl. Sci.*, vol. 7, no. 6, p. 476, Jun. 2017.

[16] M. Zheng, T. Li, R. Zhu, J. Chen, Z. Ma, M. Tang, Z. Cui and Z. Wang, "Traffic accident's severity prediction: A deep-learning approach-based CNN network," *IEEE Access*, vol. 7, pp. 39897-39910, 2019.

[17] Y. Pei, Y. Wen and S. Pan, "Road traffic accident risk prediction and key factor identification framework based on explainable deep learning," *IEEE Access*, vol. 12, pp. 120597-120611, 2024, doi: 10.1109/ACCESS.2024.3451522.

[18] N. Hamdan and T. Sipos, "Advancements in machine learning for traffic accident severity prediction: A comprehensive review," *Period. Polytech. Transp. Eng.*, vol. 53, no. 3, pp. 347-355, 2025, doi: 10.3311/PPtr.40369.

[19] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[20] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.